

Debugging Overview

Enda McGrath
Software Engineer
Developer Technical Services

Agenda

- **Debugging Tools**
  - Falcon dev environment vs PODs or Codewarrior
  - GDB
  - DDD
- **GDB & Falcon SDK**
- **GDB Commands**
- **Debugging Simulator vs Device**
- **Building & debugging application in the Simulator**
- **Remote debugging**
  - System requirements
  - NFS
  - Using the Synaptic Package Manage on Ubuntu
  - Mounting sysroot/arm-linux directory
  - Configuring environment
    - Building for arm
  - Running gdbserver & gdb
    - Connect to a running process
- **Remote debugging demo**



GDB

- Source level debugger – (Free under GPL)
  - Command line interface
    - Start your program
    - Make your program stop on specified conditions
    - Examine what has happened
    - Change things (temporally) in your program to correct problem
- Documentation
  - Developer guide - Debugging section
  - http://sourceware.org/gdb/documentation
  - Book: Debugging with GDB: The GNU Source-Level Debugger



DDD Data Display Debugger

- GNU DDD is a graphical front-end for command-line debuggers (GDB)
- http://www.gnu.org/software/ddd/ or Synaptic package manager
- Add function to the .bashrc in your home directory
- function gddd

  {

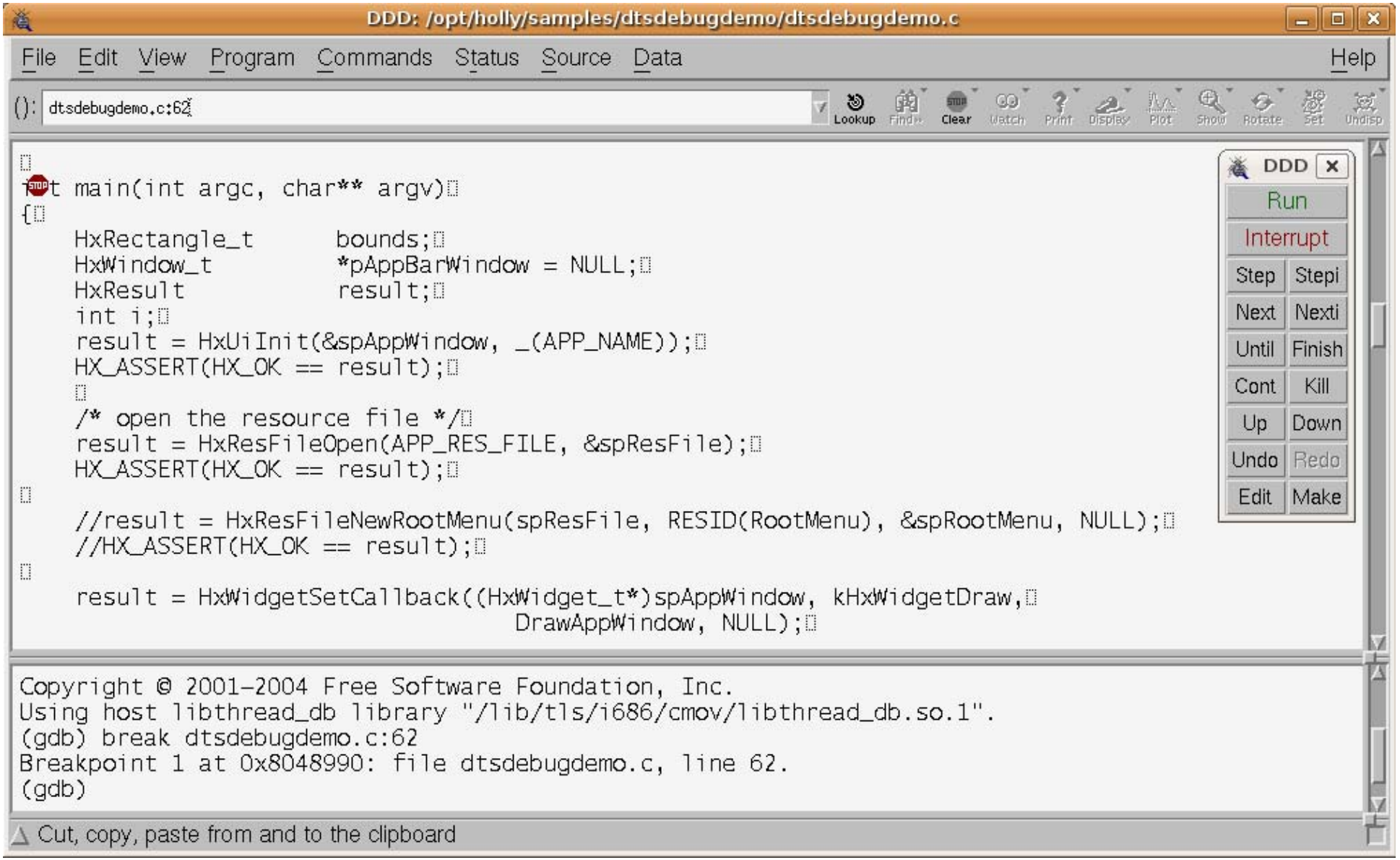
  LD_LIBRARY_PATH=/opt/holly/sysroot/i386-linux/usr/lib \

  ddd /opt/holly/sysroot/i386-linux/Applications/com.palmsdk/${1}/${1}

  } # Our SDK directory
- Call from terminal
  - $ gddd myapplication



DDD Data Display Debugger



GDB & Falcon SDK

- Installed in the /opt/holly/toolroot directory
- i386-linux-gdb
  - May be used for the simulator
- gdb
  - the workstation's gdb works fine as well
- arm-linux-gdb
  - MUST be used for arm debugging (remote debugging)
  - $ /opt/holly/toolroot/bin/arm-linux-gdb



GDB Commands – Start to finish

- Shortcut:Type enough chars for a unique command
- r - run: Start program as normal.
- b – break: set break point
  - b [line num] , b [foo.c:line num]
  - b [function name], b [foo.c:function]
- l – list: Prints source lines
- p - print variable: Display the contents of the listed variable on screen.
  - Decimal - p /d variable. Hex - p /x variable
- c - continue: Continue running the program.
- s - step: Steps forward through the program, including all subroutines.
- n - next: Steps forward through the program, bypassing all subroutines.
- info threads - information: Display a summary of the current threads.
  - info b, info line, info sharedlib, info stack. Info variables



GDB Commands - cont

- bt - backtrace: Display the current stack.
- h – help : Display the help screen.
- x - examine memory
- delete – remove all breakpoints
  - delete 95
- commands – a breakpoint can be given a series commands
  - e.g. Print a variable, set a variable – instead of recompiling
- Condition – specify expression as the break condition
- set – manually set the value of a variable while running
- q - quit: Quit the debugger.
- kill – Kills the process but maintains breakpoint settings
  - Use when program hasn't change but depends has.



Debugging difference between sim & device

- Simulator consists of Falcon's core applications and libraries compiled for i386
  - Wifi, SD, can emulated
  - Bluetooth & CF are NOT supported
- Running gdb
  - Simulator – first command is run (r), in order to start the app
  - Device – App is already running, so just continue (c)



Building & debugging application in the Simulator

- makefile
  - Compile for i386
  - Debugging info, adding command –g
  - makefile places executable in simulator path
- Use the provided gexe function

```
function gexe
{
LD_LIBRARY_PATH=/opt/holly/sysroot/i386-linux/usr/lib  \
gdb /opt/holly/sysroot/i386-linux/Applications/com.palmsdk/${1}/${1}
}
```



Demonstration…

- Debugging application in simulator
  - Setting a breakpoint
  - Deleting a breakpoint
  - Backtraces
  - Info sharedlib
  - Info threads
  - list
  - print
  - x
  - Commands and conditions
  - quit vs kill
- Simulated crash
  - Core dump & minicore stacktrace dump



Remote debugging - System requirements

- Development PC with the Falcon SDK installed
- Dedicated Ethernet card (for debugging only) in the development PC
- Crossover Ethernet cable
- Falcon device with current ROM image
- Supported USB-to-Ethernet adapter (palm tested)
  - Linksys USB100TX
  - TrendNet TU-ET100C
- NFS server running on development PC



Networking File System - NFS

- Install NFS on Ubuntu machine
  - nfs.sourceforge.net
  - Terminal apt-get
    - $ sudo apt-get update
    - $ sudo apt-get install nfs-user-server
  - Synaptic Package manager
    - System -> Administration -> Synaptic Package Manager
    - May need to update repositories
    - Search for nfs-user-server
  - RedHat
    - RPM



Mounting sysroot/arm-linux via NFS

- After installing NFS, restart portmap and nfs
  - In terminal on Ubuntu dev box
    - $ sudo /etc/init.d/portmap restart
    - $ sudo /etc/init.d/nfs-user-server restart
- On Falcon
  - Connect using cross-over cable & USB Ethernet adapter
  - Open a terminal
  - $ sume (no password)
  - # /etc.init.d/nfs start
  - Success: Mounting /mnt/nfs…done
- (Falcon)/mnt/nfs = (Ubuntu)/opt/holly/sysroot/arm-linux



Configuring for remote debugging

- On Ubuntu
  - Compile *myapplication* for arm
    - $ make arm
  - Create .gdbinit file in workstation's home directory
    - Add: set solib-absolute-prefix /opt/holly/sysroot/arm-linux
      - Note: Must be deleted (re-named) to use simulator again
- On Falcon
  - Copy *myapplication* to Falcon
    - # cp –r /mnt/nfs/Applications/com.palmsdk/myapp /Home/User/Applications/com.palmsdk
    - SD card
  - Reboot Falcon
    - Application should now be visible in menu



Running gdb and gdbserver

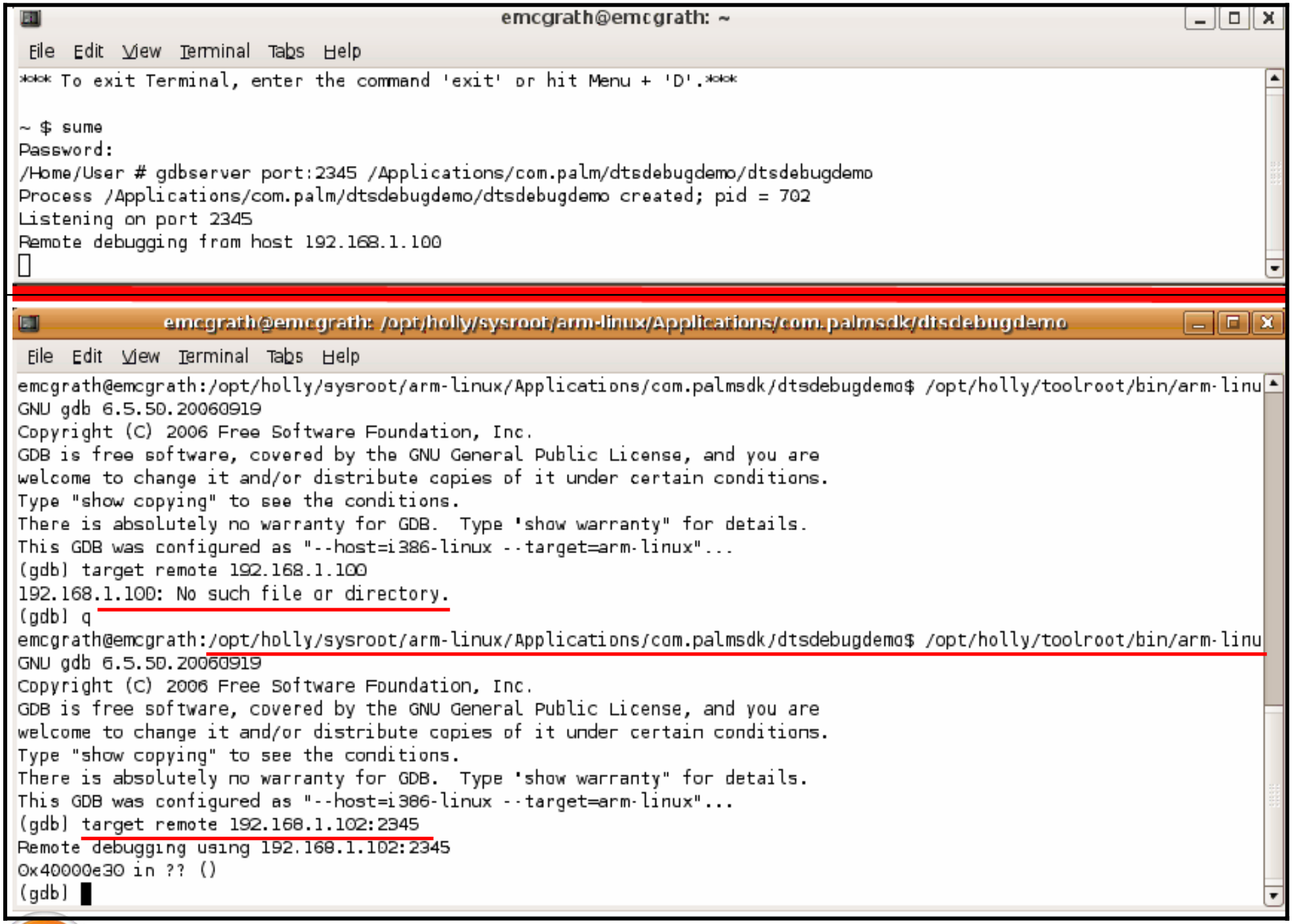
- Terminal 1 – gdbserver
  - Remote login to Falcon
    - $ ssh User@192.168.1.102 (no password)
  - $ sume (no password)
  - # gdbserver port:2345 /Applications/com.palmsdk/*myapp*/*myapp*
- Terminal 2 – gdb
  - *$* cd /opt/holly/sysroot/arm-linux/Applications/com.palmsdk/*myapp*
  - *$* /opt/holly/toolroot/bin/arm-linux-gdb *myapp*
  - (gdb) target remote 192.168.1.102:2345
- Alternatively
  - Use one terminal on dev box & one on Falcon
  - Falcon - gdbserver



Connecting to a running process

- $ ps
  - Lists currently running processes
- To connect
  - gdbserver 192.168.1.102:2345 - - attach 694
    - Where 694 is the process ID number of the first thread listed




emcgrath@emcgrath: ~
File Edit View Terminal Tabs Help
*** To exit Terminal, enter the command 'exit' or hit Menu + 'D'.***
~ $ sume
Password:
/Home/User # gdbserver port:2345 /Applications/com.palm/dtsdebugdemo/dtsdebugdemo
Process /Applications/com.palm/dtsdebugdemo/dtsdebugdemo created; pid = 702
Listening on port 2345
Remote debugging from host 192.168.1.100
emcgrath@emcgrath: /opt/holly/sysroot/arm-linux/Applications/com.palmsdk/dtsdebugdemo
File Edit View Terminal Tabs Help
emcgrath@emcgrath:/opt/holly/sysroot/arm-linux/Applications/com.palmsdk/dtsdebugdemo$ /opt/holly/toolroot/bin/arm-linu
GNU gdb 6.5.50.20060919
Copyright (C) 2006 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type 'show warranty" for details.
This GDB was configured as "--host=i386-linux --target=arm-linux"...
(gdb) target remote 192.168.1.100
192.168.1.100: No such file or directory.
(gdb) q
emcgrath@emcgrath:/opt/holly/sysroot/arm-linux/Applications/com.palmsdk/dtsdebugdemo$ /opt/holly/toolroot/bin/arm-linu
GNU gdb 6.5.50.20060919
Copyright (C) 2006 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type 'show warranty" for details.
This GDB was configured as "--host=i386-linux --target=arm-linux"...
(gdb) target remote 192.168.1.102:2345
Remote debugging using 192.168.1.102:2345
0x40000e30 in ?? ()
(gdb)

Remote debugging an application, quick demo

- (gdb) b main – set a breakpoint at main
- (gdb) c – continue
- (gdb) Info threads
- (gdb) l - list
- (gdb) n – step forward



Backup Slides

Core Dump

- Prerequisites
  - Use Compact Flash to allow for larger core dump files
- Enabling Core Dumps
  - $ sume # become the root user.
  - $ /usr/bin/enable_coredumps # enable core dumps
- Disable coredumps
  - $ sume # become the root user.
  - $ /usr/bin/enable_coredumps disable # disable core dumps
- Dumps are written to /cores/processname.time.pid.core
  - See developer guide for more detailed information

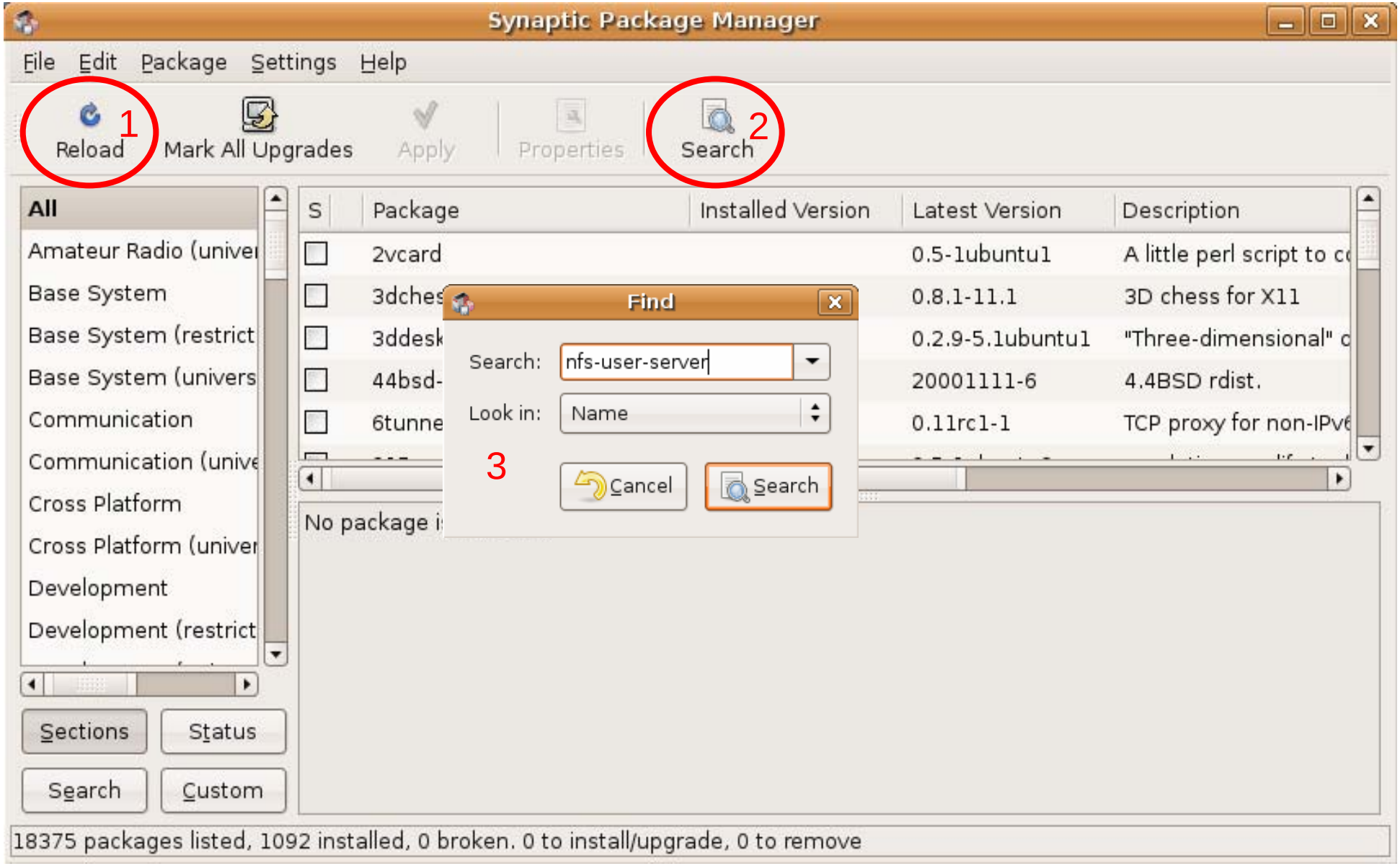


minicore stacktrace dumps

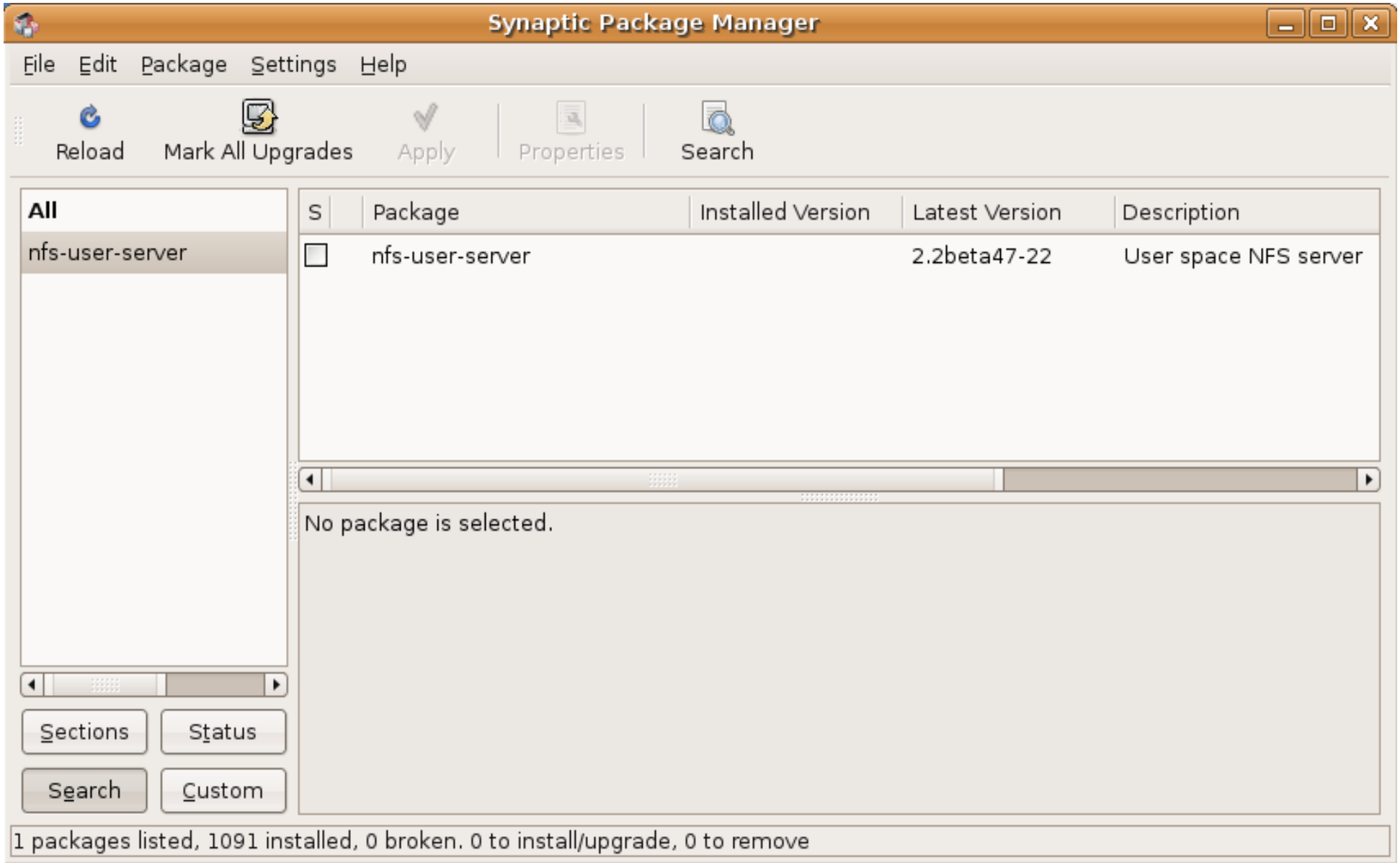
- Whenever Application X crashes, a crash dialog is shown to the user that allows the user to view stacktrace information.
- a file is also generated
  - /cores/mini/app_name.pid.minicore
- To obtain a stacktrace on the device
  - $ minicore /cores/mini/app_name.pid.minicore
- To access the gui again, you may do
  - $ minicore -g /cores/mini/app_name.pid.minicore



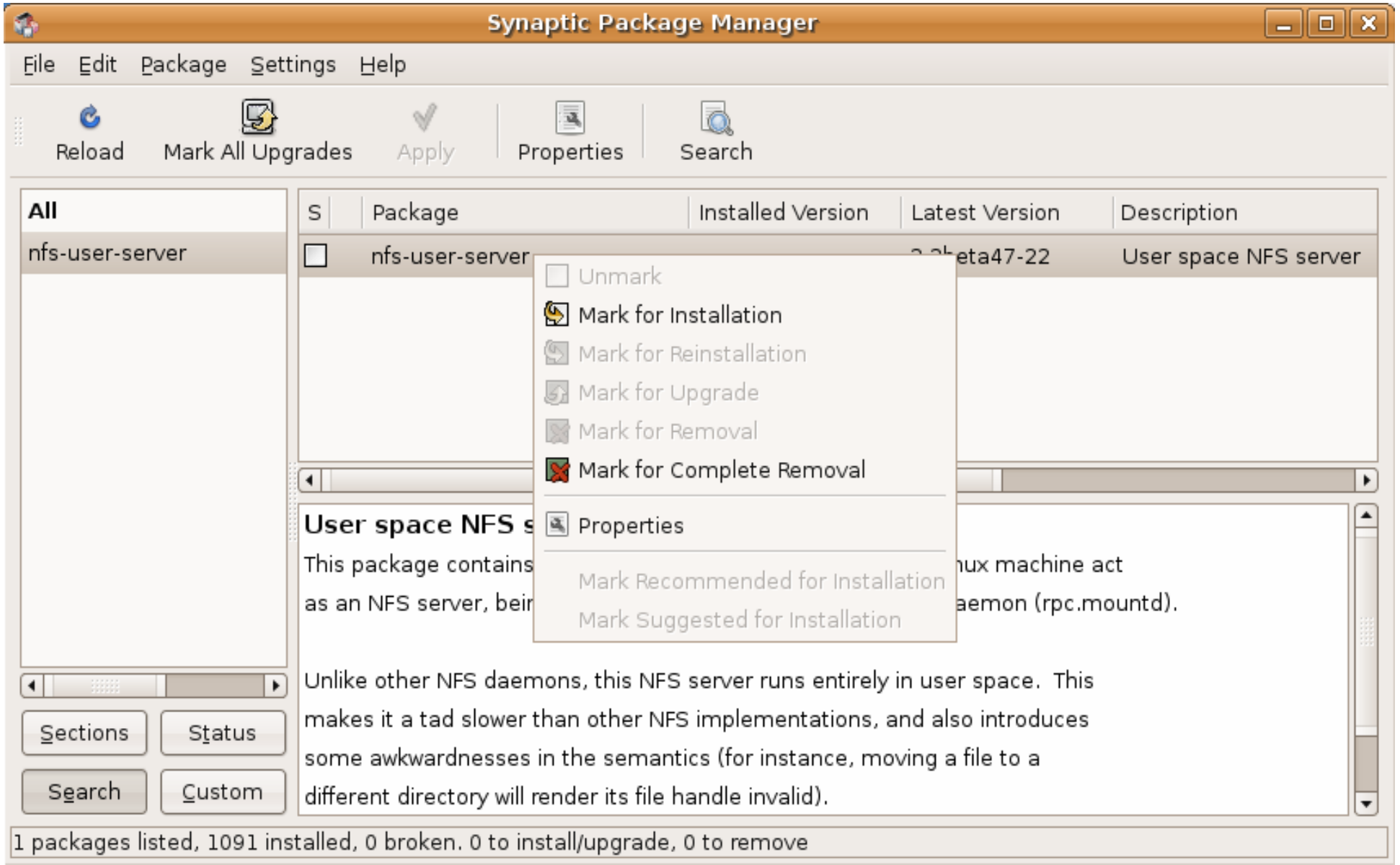
NFS – Synaptic Package Manager



NFS – Synaptic Package Manager



NFS – Synaptic Package Manager



NFS – Synaptic Package Manager

